

JARVIS AI — Complete Setup Guide

Kali Linux Local + Cloud Deployment

Repository: <https://github.com/mikesamuel2814/jarvis-ai.git>

Version: 1.0 | **Date:** June 2026

Target: Kali Linux (Rolling) + Modal/RunPod Cloud GPU

Table of Contents

- 1. Prerequisites
- 2. Kali Linux Local Setup
- 3. Cloud GPU Setup
- 4. GitHub Repository Integration
- 5. Kimi CLI Integration
- 6. JARVIS Core Installation
- 7. Model Training & Deployment
- 8. Security Hardening
- 9. Verification & Testing
- 10. Troubleshooting

1. Prerequisites

1.1 Hardware Requirements

Component	Minimum	Recommended
GPU	NVIDIA GTX 1660 (6GB)	RTX 4090 (24GB) / A100 (80GB)
RAM	16GB	64GB DDR5
Storage	100GB SSD	1TB NVMe
CPU	8-core	16-core (AMD Ryzen 9 / Intel i9)
Network	50 Mbps	1 Gbps (for cloud sync)

1.2 Kali Linux Base Requirements

```
# Verify Kali version
cat /etc/os-release
# Expected: Kali GNU/Linux Rolling (2026.x)

# Update system (critical on Kali)
sudo apt update && sudo apt full-upgrade -y

# Install base build dependencies
sudo apt install -y build-essential git curl wget vim htop tmux
↪ software-properties-common apt-transport-https ca-certificates gnupg lsb-release
```

1.3 NVIDIA GPU Setup (Kali)

Kali uses Debian-based repositories but requires manual NVIDIA driver installation:

```
# 1. Disable Nouveau
echo "blacklist nouveau
options nouveau modeset=0" | sudo tee /etc/modprobe.d/blacklist-nouveau.conf
sudo update-initramfs -u

# 2. Install kernel headers
sudo apt install -y linux-headers-$(uname -r)

# 3. Add NVIDIA repository (use Debian method for Kali)
wget https://developer.download.nvidia.com/compute/cuda/repos/debian12/x86_64/
↪ cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update

# 4. Install CUDA Toolkit (includes drivers)
sudo apt install -y nvidia-driver nvidia-cuda-toolkit

# 5. Reboot
sudo reboot

# 6. Verify
nvidia-smi
nvcc --version
```

Note: If `nvidia-smi` fails after reboot, Kali may need the `nvidia-persistenced` daemon:

```
sudo systemctl enable nvidia-persistenced
sudo systemctl start nvidia-persistenced
```

2. Kali Linux Local Setup

2.1 Python Environment (Isolated)

Do NOT use system Python on Kali. Kali's system Python is heavily modified for security tools.

```
# Install pyenv dependencies
sudo apt install -y make libssl-dev zlib1g-dev libbz2-dev      libreadline-dev libsqlite3-dev llvm
↪ libncursesw5-dev      xz-utils tk-dev libxml2-dev libxmlsec1-dev libffi-dev liblzma-dev

# Install pyenv
curl https://pyenv.run | bash

# Add to ~/.zshrc (or ~/.bashrc)
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
echo 'eval "$(pyenv init -)"' >> ~/.zshrc
source ~/.zshrc
```

```
# Install Python 3.11 (stable for ML)
pyenv install 3.11.9
pyenv global 3.11.9

# Verify
python --version # Python 3.11.9
which python     # ~/.pyenv/shims/python
```

2.2 Project Directory Structure

```
# Create workspace
mkdir -p ~/jarvis-ai/{data,models,src,scripts,config,logs}
cd ~/jarvis-ai

# Clone your repository
git clone https://github.com/mikesamuel2814/jarvis-ai.git .

# Set proper permissions (Kali often runs as root — create dedicated user)
# If running as root (not recommended for model serving):
useradd -m -s /bin/bash jarvis
chown -R jarvis:jarvis ~/jarvis-ai
su - jarvis
cd ~/jarvis-ai
```

2.3 Virtual Environment

```
python -m venv venv-jarvis
source venv-jarvis/bin/activate

# Upgrade base tools
pip install --upgrade pip setuptools wheel
```

2.4 Core Dependencies (Kali-Optimized)

```
# Create requirements-kali.txt
cat > requirements-kali.txt << 'EOF'

# Core ML / Training
torch==2.3.1+cu121
unsloth[cu121-ampere] @ git+https://github.com/unslothai/unsloth.git
transformers>=4.42.0
datasets>=2.20.0
accelerate>=0.32.0
peft>=0.11.0
bitsandbytes>=0.43.0
trl>=0.15.0
axolotl[flash-attn,deepspeed] @ git+https://github.com/OpenAccessMachineLearning/axolotl.git

# Agent / Memory
langgraph>=0.2.0
```

```

langchain>=0.2.0
langchain-community>=0.2.0
llama-index>=0.11.0
chromadb>=0.5.0
qdrant-client>=1.10.0

# Serving
vllm>=0.5.0
llama-cpp-python>=0.2.85

# Utilities
pydantic>=2.7.0
aiosqlite>=0.20.0
aiohttp>=3.9.0
fastapi>=0.111.0
uvicorn>=0.30.0
python-multipart>=0.0.9

# Security / Monitoring (Kali-native)
psutil>=6.0.0
cryptography>=42.0.0
python-jose>=3.3.0
EOF

# Install with CUDA 12.1 support (match Kali's CUDA)
pip install -r requirements-kali.txt --extra-index-url https://download.pytorch.org/whl/cu121

# Verify PyTorch CUDA
python -c "import torch; print(f'CUDA: {torch.cuda.is_available()}, Device:
↳ {torch.cuda.get_device_name(0)}')"

```

2.5 Kali-Specific Security Configuration

```

# 1. Firewall: Allow only local API access
sudo apt install -y ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 127.0.0.1 to any port 8000 # JARVIS API
sudo ufw allow from 127.0.0.1 to any port 11434 # Ollama (if used)
sudo ufw enable

# 2. Create dedicated service user
sudo useradd -r -s /bin/false -M jarvis-service
sudo mkdir -p /var/lib/jarvis
sudo chown jarvis-service:jarvis-service /var/lib/jarvis

# 3. AppArmor profile (optional but recommended)
sudo apt install -y apparmor-utils
# Create profile at /etc/apparmor.d/usr.local.bin.jarvis

```

2.6 Systemd Service (Auto-start JARVIS)

Create `/etc/systemd/system/jarvis.service`:

```
[Unit]
Description=JARVIS AI Personal Assistant
After=network.target nvidia-persistenced.service
Wants=nvidia-persistenced.service

[Service]
Type=simple
User=jarvis-service
Group=jarvis-service
WorkingDirectory=/home/jarvis/jarvis-ai
Environment="PATH=/home/jarvis/jarvis-ai/venv-jarvis/bin"
Environment="CUDA_VISIBLE_DEVICES=0"
Environment="JARVIS_MODE=local"
ExecStart=/home/jarvis/jarvis-ai/venv-jarvis/bin/python scripts/serve.py --model models/final/
↳ jarvis-dpo
Restart=always
RestartSec=10
StandardOutput=append:/var/log/jarvis/jarvis.log
StandardError=append:/var/log/jarvis/jarvis-error.log

[Install]
WantedBy=multi-user.target
```

Enable:

```
sudo mkdir -p /var/log/jarvis
sudo chown jarvis-service:jarvis-service /var/log/jarvis
sudo systemctl daemon-reload
sudo systemctl enable jarvis
sudo systemctl start jarvis
sudo systemctl status jarvis
```

3. Cloud GPU Setup

3.1 Platform Selection

Platform	Best For	Cost/hr (A100)	Setup Complexity
Modal	Serverless training bursts	~\$2.50	Low
RunPod	On-demand GPU pods	~\$1.99	Medium
Lambda Labs	Persistent training	~\$1.60	Low
Vast.ai	Budget spot instances	~\$0.80	High

Recommendation: Use **Modal** for training automation, **RunPod** for manual experimentation.

3.2 Modal (Serverless) Setup

```
# 1. Install Modal CLI
pip install modal

# 2. Authenticate
modal token new
# Follow browser authentication

# 3. Create modal config
cat > modal_train.py << 'EOF'
import modal

image = modal.Image.debian_slim(python_version="3.11") \
    .apt_install("git", "build-essential",
    ↪ "wget") \
    .pip_install(
        "torch==2.3.1+cu121",
        "unsloth",
        "transformers",
        "datasets",
        "trl",
        "axolotl",
        "accelerate",
        "peft",
        "bitsandbytes",
        extra_index_url="https://download.pytorch.org/whl/cu121"
    ) \
    .run_commands(
        "huggingface-cli download deepseek-ai/DeepSeek-R1-Distill-Qwen-7B --local-dir /model"
    )

app = modal.App("jarvis-training")

@app.function(
    gpu="A100-80GB",
    image=image,
    timeout=86400, # 24 hours
    volumes={"/data": modal.Volume.from_name("jarvis-data")}
)
def train_jarvis(config_yaml: str):
    import subprocess
    subprocess.run(["axolotl", "train", config_yaml], check=True)
    return "Training complete"

@app.local_entrypoint()
def main():
    result = train_jarvis.remote("/data/configs/jarvis_sft.yaml")
    print(result)
EOF

# 4. Deploy training
modal run modal_train.py
```

3.3 RunPod Setup

```
# 1. Install RunPod CLI
pip install runpod

# 2. Set API key
runpod config --api-key YOUR_API_KEY

# 3. Create pod template
cat > runpod_template.json << 'EOF'
{
  "name": "jarvis-training",
  "imageName": "runpod/pytorch:2.2.0-py3.11-cuda12.1-devel-ubuntu22.04",
  "gpuType": "NVIDIA RTX A100 80GB",
  "cloudType": "SECURE",
  "volumeSize": 100,
  "containerDiskSize": 50,
  "ports": "8000/http,22/tcp",
  "env": [
    {"key": "JARVIS_MODE", "value": "cloud"},
    {"key": "HF_TOKEN", "value": "YOUR_HF_TOKEN"}
  ]
}
EOF

# 4. Create pod
runpod pod create --config runpod_template.json

# 5. SSH into pod and setup
ssh root@<pod-ip> -p <port>
# Inside pod:
git clone https://github.com/mikesamuel2814/jarvis-ai.git
cd jarvis-ai
pip install -r requirements-kali.txt
python scripts/train.py
```

3.4 Cloud-to-Local Sync

```
# After cloud training, sync model back to Kali
# Option A: HuggingFace Hub (recommended)
huggingface-cli upload mikesamuel2814/jarvis-dpo models/checkpoints/jarvis-dpo .

# On Kali:
huggingface-cli download mikesamuel2814/jarvis-dpo --local-dir models/final/

# Option B: Direct rsync (if VPN/peer connection)
rsync -avz --progress root@<cloud-ip>:/root/jarvis-ai/models/checkpoints/ ~/jarvis-ai/models/cloud/

# Option C: Modal Volume export
modal volume get jarvis-data models/checkpoints/jarvis-dpo ./models/final/
```

4. GitHub Repository Integration

4.1 Repository Structure (Standardized)

Your repo <https://github.com/mikesamuel2814/jarvis-ai.git> should follow this structure:

```
jarvis-ai/
├── .github/
│   ├── workflows/
│   │   ├── train-cloud.yml    # GitHub Actions → Modal/RunPod
│   │   └── test-local.yml     # CI tests on Kali
│   └── data/
│       ├── raw/               # .gitignored personal files
│       ├── synthetic/         # Generated training data
│       └── configs/           # Axolotl YAML files
├── models/
│   ├── base/                 # .gitignored (large files)
│   └── final/                # .gitignored (large files)
├── src/
│   ├── agent/                # LangGraph state machine
│   ├── tools/                # Tool definitions + executors
│   ├── memory/               # ChromaDB + SQLite stores
│   ├── safety/               # Constitution + filters
│   └── synthetic/            # Data generation pipeline
├── scripts/
│   ├── train.py              # Master training script
│   ├── generate_data.py      # Synthetic data generator
│   ├── serve.py              # vLLM serving
│   └── evaluate.py           # Benchmark suite
├── config/
│   ├── jarvis_sft.yml        # Axolotl SFT config
│   ├── jarvis_dpo.yml        # Axolotl DPO config
│   └── kali-security.conf     # Kali hardening rules
├── requirements-kali.txt      # Kali dependencies
├── requirements-cloud.txt     # Cloud (lighter) dependencies
├── Dockerfile                # Container build
├── modal_train.py             # Modal serverless script
├── README.md
└── .gitignore
```

4.2 Git Configuration (Kali)

```
cd ~/jarvis-ai

# Set identity
git config user.name "Mike Samuel"
git config user.email "your@email.com"

# Large file handling (models)
git lfs install
```



```

git lfs track "models/final/*.gguf"
git lfs track "models/final/*.safetensors"

# Create .gitignore
cat > .gitignore << 'EOF'
# Models & Data
models/base/
models/checkpoints/
*.bin
*.gguf
*.safetensors
data/raw/
*.db
*.sqlite

# Python
__pycache__/
*.py[cod]
*$py.class
venv*/
.env

# Logs
logs/
*.log

# IDE
.vscode/
.idea/
*.swp
EOF

git add .
git commit -m "JARVIS v1.0: Kali + Cloud setup"
git push origin main

```

4.3 GitHub Actions CI/CD

Create `.github/workflows/train-cloud.yml`:

```

name: JARVIS Cloud Training

on:
  push:
    branches: [main]
    paths:
      - 'data/configs/**'
      - 'src/synthetic/**'
  workflow_dispatch:

jobs:

```

```

train:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Setup Modal
      run: |
        pip install modal
        modal token set ${ secrets.MODAL_TOKEN }

    - name: Trigger Training
      run: modal run modal_train.py

    - name: Download Model
      run: modal volume get jarvis-data models/checkpoints/jarvis-dpo ./models/cloud/

    - name: Upload Artifact
      uses: actions/upload-artifact@v4
      with:
        name: jarvis-model
        path: models/cloud/

```

5. Kimi CLI Integration

5.1 Kimi CLI Setup (Already Installed)

Verify and configure:

```

# Check installation
kimi --version

# Login (if not already)
kimi login

# Set default model for JARVIS context
kimi config set model kimi-k2.6
kimi config set output_format json

```

5.2 Using Kimi CLI for Synthetic Data Generation

```

# Create Kimi-powered data generator script
cat > scripts/kimi_generate.sh << 'EOF'
#!/bin/bash
# Generate synthetic training data using Kimi CLI

PERSONA="You are JARVIS data generator. Create expert-level training examples."
OUTPUT_DIR="data/synthetic"
mkdir -p $OUTPUT_DIR

# Generate reasoning problems

```

```

for i in {1..100}; do
    kimi chat          --system "$PERSONA"          --message "Generate a complex system administration
↪   problem requiring multi-step reasoning. Output as JSON: {instruction, reasoning_trace, answer}"
↪   >> $OUTPUT_DIR/reasoning_$(i).jsonl
done

# Generate tool-calling examples
for tool in file_read shell_exec web_search; do
    kimi chat          --system "$PERSONA"          --message "Generate 50 training examples for the $tool
↪   tool. Format: {instruction, tool_call_json, expected_output}"          >> $OUTPUT_DIR/
↪   tools_$(tool).jsonl
done

echo "Synthetic data generation complete"
EOF

chmod +x scripts/kimi_generate.sh

```

5.3 Kimi CLI as Teacher for Distillation

```

# Use Kimi CLI to generate soft labels for knowledge distillation
cat > scripts/kimi_distill.sh << 'EOF'
#!/bin/bash
# Knowledge distillation using Kimi CLI as teacher

INPUT_DATA="data/processed/train.jsonl"
OUTPUT_DATA="data/processed/train_distilled.jsonl"

while IFS= read -r line; do
    instruction=$(echo $line | jq -r '.instruction')

    # Get teacher response (soft label)
    teacher_output=$(kimi chat          --system "You are an expert AI assistant. Think step by step."
↪   --message "$instruction")

    # Append to dataset
    echo "$line" | jq --arg output "$teacher_output" '. + {teacher_output: $output}' >> $OUTPUT_DATA

    sleep 1 # Rate limit
done < "$INPUT_DATA"
EOF

chmod +x scripts/kimi_distill.sh

```

5.4 Integration with JARVIS Agent

```

# src/tools/kimi_bridge.py
import subprocess
import json

class KimiBridge:

```

```

'''Bridge to Kimi CLI for tasks requiring frontier model capability.'''

def __init__(self, model: str = "kimi-k2.6"):
    self.model = model

def generate(self, prompt: str, system: str = "", max_tokens: int = 2000) -> str:
    '''Call Kimi CLI and return response.'''
    cmd = [
        "kimi", "chat",
        "--model", self.model,
        "--message", prompt
    ]
    if system:
        cmd.extend(["--system", system])

    result = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
        timeout=120
    )

    if result.returncode != 0:
        return f"Kimi CLI Error: {result.stderr}"

    return result.stdout

def generate_structured(self, prompt: str, schema: dict) -> dict:
    '''Generate structured output via Kimi.'''
    structured_prompt = f"""{prompt}

You must respond with valid JSON matching this schema:
{json.dumps(schema, indent=2)}

Respond ONLY with JSON."""

    response = self.generate(structured_prompt)

    # Extract JSON from response
    try:
        # Find JSON block
        start = response.find('{')
        end = response.rfind('}') + 1
        if start != -1 and end != -1:
            return json.loads(response[start:end])
    except:
        pass

    return {"error": "Failed to parse", "raw": response}

```

6. JARVIS Core Installation

6.1 Automated Setup Script

Create scripts/setup.sh:

```
#!/bin/bash
set -e

echo "┌───────────────────────────────────┐"
echo "│           JARVIS AI Setup Script           │"
echo "│           Kali Linux + Cloud Compatible       │"
echo "└───────────────────────────────────┘"

# Detect environment
if [ -f /etc/os-release ]; then
    . /etc/os-release
    OS=$NAME
else
    OS=$(uname -s)
fi

if command -v nvidia-smi &> /dev/null; then
    GPU=$(nvidia-smi --query-gpu=name --format=csv,noheader | head -n1)
    echo "OS: $OS | GPU: $GPU"
else
    echo "OS: $OS | GPU: None detected"
fi

# 1. Python check
if ! command -v python3.11 &> /dev/null; then
    echo "Installing Python 3.11..."
    if [[ "$OS" == *"Kali"* ]] || [[ "$OS" == *"Debian"* ]]; then
        sudo apt update
        sudo apt install -y python3.11 python3.11-venv python3.11-dev
    else
        echo "Please install Python 3.11 manually"
        exit 1
    fi
fi

# 2. Virtual environment
if [ ! -d "venv-jarvis" ]; then
    echo "Creating virtual environment..."
    python3.11 -m venv venv-jarvis
fi

source venv-jarvis/bin/activate

# 3. Dependencies
echo "Installing dependencies..."
if [[ "$GPU" == *"NVIDIA"* ]]; then
```

```

    pip install -r requirements-kali.txt
else
    pip install -r requirements-cloud.txt
fi

# 4. Model download
echo "Downloading base model..."
if [ ! -d "models/base/deepseek-r1-qwen-7b" ]; then
    huggingface-cli download deepseek-ai/DeepSeek-R1-Distill-Qwen-7B --local-dir models/base/
    ↪ deepseek-r1-qwen-7b --local-dir-use-symlinks False
fi

# 5. Data directories
mkdir -p data/{raw,synthetic,processed,configs}
mkdir -p models/{checkpoints,final}
mkdir -p logs

# 6. Git hooks (optional)
if [ -d ".git" ]; then
    cat > .git/hooks/pre-commit << 'HOOK'
#!/bin/bash
# Prevent committing large models
if git diff --cached --name-only | grep -E '\.(bin|gguf|safetensors)$'; then
    echo "Blocked: Do not commit model files. Use Git LFS or HuggingFace."
    exit 1
fi
HOOK
    chmod +x .git/hooks/pre-commit
fi

echo "Setup complete!"
echo ""
echo "Next steps:"
echo " 1. Generate data: ./scripts/kimi_generate.sh"
echo " 2. Train model: python scripts/train.py"
echo " 3. Serve JARVIS: python scripts/serve.py --interactive"
```

Run:

```

chmod +x scripts/setup.sh
./scripts/setup.sh
```

7. Model Training & Deployment

7.1 Training Execution Matrix

Environment	Command	Duration	Cost
Kali Local	python scripts/ train.py	12-24h	Electricity only

Table 3 – continued

Environment	Command	Duration	Cost
Modal Cloud	modal run modal_train.py	4-8h	~\$15-30
RunPod	python scripts/ train.py (inside pod)	4-8h	~\$8-16
Kimi CLI Assisted	./scripts/ kimi_distill.sh	24-48h	API credits

7.2 Training Pipeline (Single Command)

```
# Full automated pipeline
python scripts/train.py

# Steps executed:
# 1. Hardware detection & config optimization
# 2. SFT training (Axolotl + Unsloth)
# 3. LoRA merge
# 4. DPO alignment
# 5. Export to GGUF (4-bit)
# 6. Validation inference
```

7.3 Deployment Modes

Mode A: Local API (Kali)

```
# vLLM serving
python scripts/serve.py --model models/final/jarvis-dpo

# Test
curl http://localhost:8000/chat -H "Content-Type: application/json" -d '{"message": "Hello JARVIS",
↵  "session_id": "test"}'
```

Mode B: Interactive Terminal

```
python scripts/serve.py --model models/final/jarvis-dpo --interactive
```

Mode C: Cloud Endpoint

```
# On RunPod/Modal, expose public endpoint
python scripts/serve.py --model models/final/jarvis-dpo --host 0.0.0.0 --port 8000
```

8. Security Hardening

8.1 Kali-Specific Hardening

```
# 1. Disable unnecessary Kali services
sudo systemctl disable bluetooth
sudo systemctl disable avahi-daemon
```

```

# 2. Restrict JARVIS file access
sudo setfacl -R -m u:jarvis-service:rx /home/jarvis/jarvis-ai
sudo setfacl -R -m u:jarvis-service:r /var/log

# 3. Network isolation (if multi-user)
sudo iptables -A INPUT -p tcp --dport 8000 -s 127.0.0.1 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 8000 -j DROP
sudo iptables-save | sudo tee /etc/iptables/rules.v4

# 4. Audit JARVIS actions
sudo apt install -y auditd
sudo auditctl -w /home/jarvis/jarvis-ai/src/tools/ -p rwx -k jarvis-tools
sudo auditctl -w /var/log/jarvis/ -p wa -k jarvis-logs

```

8.2 JARVIS Safety Constitution (Runtime)

```

# config/safety_rules.json
{
  "absolute_blocks": [
    {
      "pattern": "rm\s+-rf\s+/",
      "action": "block",
      "severity": "critical"
    },
    {
      "pattern": "mkfs\.",
      "action": "block",
      "severity": "critical"
    },
    {
      "pattern": "wget.*\|.*sh",
      "action": "confirm",
      "severity": "high"
    }
  ],
  "resource_limits": {
    "max_file_size_mb": 100,
    "max_command_timeout": 300,
    "blocked_paths": [
      "/etc/shadow",
      "/root/.ssh",
      "/boot"
    ]
  }
}

```


9. Verification & Testing

9.1 Health Check Script

```
cat > scripts/health_check.sh << 'EOF'
#!/bin/bash

echo "JARVIS Health Check"

# 1. Python environment
python --version || exit 1

# 2. CUDA
python -c "import torch; assert torch.cuda.is_available(), 'CUDA not available'" && echo "CUDA OK" ||
↪ echo "CUDA FAIL"

# 3. Model files
[ -d "models/base/deepseek-r1-qwen-7b" ] && echo "Base model OK" || echo "Base model missing"
[ -f "models/final/jarvis-q4_k_m.gguf" ] && echo "Final model OK" || echo "Final model not built yet"

# 4. Dependencies
python -c "import unsloth, trl, vllm, langgraph" && echo "Core packages OK" || echo "Missing packages"

# 5. Disk space
DF=$(df -h . | tail -1 | awk '{print $4}')
echo "Free space: $DF"

# 6. Memory
FREE=$(free -h | grep Mem | awk '{print $7}')
echo "Free RAM: $FREE"

# 7. GPU VRAM
if command -v nvidia-smi &> /dev/null; then
    VRAM=$(nvidia-smi --query-gpu=memory.free --format=csv,noheader,nounits | head -n1)
    echo "Free VRAM: ${VRAM}MB"
fi

echo "Health check complete"
EOF

chmod +x scripts/health_check.sh
./scripts/health_check.sh
```

9.2 Benchmark Suite

```
# Run evaluation
python scripts/evaluate.py --model models/final/jarvis-dpo --tests eval/test_cases.json

# Expected minimum scores to beat free LLMs:
# Reasoning: > 70%
# Tool use: > 80%
# Safety: 100%
```

```
# Latency: < 500ms per token
```

10. Troubleshooting

10.1 Kali-Specific Issues

Issue	Cause	Fix
nvidia-smi not found	Driver not installed	Reinstall CUDA: <code>sudo apt install --reinstall nvidia-driver</code>
PyTorch CUDA false	Wrong PyTorch version	<code>pip install torch --force-reinstall --index-url https://download.pytorch.org/whl/cu121</code>
Permission denied on models	Running as non-root	<code>sudo chown -R \$USER:\$USER ~/jarvis-ai</code>
apt conflicts	Kali rolling release	<code>sudo apt --fix-broken install</code> then <code>sudo apt full-upgrade</code>
OOM during training	VRAM exhausted	Reduce <code>max_seq_length</code> to 4096, increase <code>gradient_accumulation_steps</code>
Kimi CLI not found	PATH issue	which kimi or reinstall: <code>pip install kimi-cli</code>

10.2 Cloud-Specific Issues

Issue	Cause	Fix
Modal timeout	Training > 24h	Increase <code>timeout=172800</code> (48h) or use checkpointing
RunPod pod terminated	Spot instance reclaimed	Use SECURE cloud type, not COMMUNITY
HF download fails	Token missing	<code>export HF_TOKEN=xxx</code> or <code>huggingface-cli login</code>
Slow cloud sync	Large model files	Use <code>huggingface-cli upload</code> instead of <code>rsync</code>

10.3 Performance Tuning

```
# If training is slow on Kali:
# 1. Enable CPU performance mode
sudo apt install -y cpufrequtils
sudo systemctl enable cpufrequtils
```

```
# 2. Set NVIDIA persistence mode
sudo nvidia-smi -pm 1

# 3. Disable GPU watchdog (if stable)
export PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:512

# 4. Use Flash Attention 2
pip install flash-attn --no-build-isolation

# In config: flash_attention: true
```

Appendix A: Quick Reference Card

```
# --- SETUP ---
./scripts/setup.sh           # Full environment setup
./scripts/health_check.sh    # Verify installation

# --- DATA ---
./scripts/kimi_generate.sh   # Generate synthetic data
python src/synthetic/filter.py # Quality filter
python scripts/prepare_data.py # Merge datasets

# --- TRAIN ---
python scripts/train.py      # Full pipeline (SFT -> DPO -> Export)
axolotl train data/configs/jarvis_sft.yml # Manual SFT
axolotl train data/configs/jarvis_dpo.yml # Manual DPO

# --- CLOUD ---
modal run modal_train.py     # Serverless training
runpod pod create --config ... # GPU pod

# --- SERVE ---
python scripts/serve.py --interactive # Terminal mode
python scripts/serve.py --host 0.0.0.0 # API mode
sudo systemctl start jarvis          # Systemd service

# --- UTILS ---
nvidia-smi           # GPU status
htop                  # CPU/RAM status
df -h                 # Disk space
journalctl -u jarvis -f # Live logs
```

Appendix B: File Manifest

File	Purpose	Environment
requirements-kali.txt	Kali dependencies	Local
requirements-cloud.txt	Lightweight cloud deps	Cloud

Table 6 – continued

File	Purpose	Environment
scripts/setup.sh	Automated setup	Both
scripts/train.py	Master training	Both
scripts/serve.py	Model serving	Both
scripts/kimi_generate.sh	Synthetic data	Both
modal_train.py	Modal serverless	Cloud
runpod_template.json	RunPod config	Cloud
config/jarvis_sft.yml	SFT training config	Both
config/jarvis_dpo.yml	DPO training config	Both
src/agent/graph.py	LangGraph agent	Both
src/tools/executors.py	Tool execution	Both
src/memory/store.py	Vector + structured memory	Both
src/safety/constitution.py	Safety rules	Both

End of Document

For updates, check: <https://github.com/mikesamuel2814/jarvis-ai.git>